
Indicaciones generales:

- Este Proyecto 1 de laboratorio es para implementarlo usando Programación Estructurada en C++. No utilice clases, ni POO.
 - Se deben desarrollar en grupos de **5 estudiantes**.
 - Los archivos se subirán directamente a www.gradescope.com. Después de entregar sus archivos, debe registrar en Gradescope a los integrantes del grupo.
-

PROYECTO: TORNEO DE ESTRATEGIAS DEL DILEMA DEL PRISIONERO

1. Descripción del proyecto

El **Dilema del Prisionero** es un problema clásico de la teoría de juegos en el que dos participantes deben decidir, de manera independiente, entre **cooperar** o **traicionar** a su oponente.

En este proyecto se desarrollará en **C++** una simulación computacional del Dilema del Prisionero Iterado. El programa deberá implementar diferentes estrategias de juego y hacerlas competir entre sí durante múltiples rondas. Cada estrategia deberá tomar decisiones basándose en su comportamiento definido y, en algunos casos, en el historial de jugadas del oponente. Además cada equipo deberá diseñar **dos estrategias propias**, las cuales también participarán en el torneo.

Al finalizar el torneo, el programa deberá calcular las estadísticas obtenidas por cada estrategia, generar un ranking, determinar cuál fue la estrategia con mayor puntuación y analizar los resultados

2. Especificaciones

2.1. Reglas del Dilema del Prisionero

En cada ronda participan dos estrategias: Jugador A y Jugador B

Cada jugador debe elegir una de las siguientes acciones:

C: Cooperar

T: Traicionar

Los puntos obtenidos dependen de las decisiones de ambos jugadores.

Matriz de puntos

Jugador A	Jugador B	Puntos A	Puntos B
C	C	3	3
C	T	0	5
T	C	5	0
T	T	1	1

2.2. Partida

Una partida consiste en enfrentar dos estrategias durante un número determinado de rondas, cada equipo decide la cantidad de rondas, la cual tiene que ser mayor a 200.

En cada ronda, cada estrategia elige su opción, y se les asigna los puntos, al finalizar las rondas se sumarán todos los puntos obtenidos por cada estrategia.

2.3. Estrategias obligatorias

Cada equipo deberá implementar las siguientes **6 estrategias**.

Estrategia 1: Siempre Cooperar, esta estrategia siempre elige: C

Estrategia 2: Siempre Traicionar, esta estrategia siempre elige: T

Estrategia 3: Aleatoria, esta estrategia deberá decidir aleatoriamente entre: C y T

Estrategia 4: Tit for Tat, esta estrategia utiliza el siguiente comportamiento:

1. En la primera ronda siempre coopera.
2. A partir de la segunda ronda, copia la decisión tomada por el oponente en la ronda anterior.

Estrategia 5: Vengativa, esta estrategia comienza cooperando. Mientras el oponente nunca haya traicionado, continúa cooperando. Sin embargo, si el oponente traiciona al menos una vez, la estrategia comenzará a traicionar en todas las rondas siguientes.

Estrategia 6: Ojo por Dos Ojos, esta estrategia comienza cooperando. A partir de la tercera ronda:

- Si el oponente traicionó en las dos rondas anteriores, traiciona.
- En cualquier otro caso, coopera.

2.4. Estrategias propias

Además de las seis estrategias obligatorias, cada equipo deberá diseñar e implementar **2 estrategias propias**. Las estrategias propias deben tener un comportamiento diferente de las estrategias obligatorias. No se aceptará simplemente cambiar el nombre de una estrategia existente.

Cada estrategia propia deberá tener:

- Nombre.
- Descripción.
- Reglas de comportamiento.
- Algoritmo utilizado.
- Justificación del diseño.

2.5. Torneo

Todas las estrategias deberán competir entre sí. Cada estrategia deberá enfrentarse contra todas las demás estrategias **una sola vez**.

No será necesario realizar:

E1 vs E2

E2 vs E1

porque representan el mismo enfrentamiento desde el punto de vista del torneo.

Los puntos obtenidos en cada ronda se acumularán durante toda la partida. Posteriormente, los puntos obtenidos en todas las partidas se acumularán para obtener la puntuación total de cada estrategia.

Por ejemplo:

Estrategia: Tit for Tat

Contra Siempre Cooperar:	1200 puntos
Contra Siempre Traicionar:	500 puntos
Contra Aleatoria:	950 puntos
Contra Vengativa:	1350 puntos
...	

PUNTOS TOTALES: 8450

Al finalizar el torneo, el programa deberá ordenar las estrategias de acuerdo con los puntos obtenidos. La estrategia con mayor cantidad de puntos será considerada:

GANADORA DEL TORNEO.

2.6. Estadísticas obligatorias

El programa deberá calcular y mostrar como mínimo las siguientes estadísticas para cada estrategia:

- Cantidad total de puntos obtenidos durante todo el torneo.
- Cantidad de partidas en las que obtuvo más puntos que su oponente.
- Cantidad de partidas en las que obtuvo menos puntos que su oponente.
- Cantidad de partidas en las que ambas estrategias obtuvieron la misma cantidad de puntos.
- Cantidad total de veces que la estrategia eligió: C
- Cantidad total de veces que la estrategia eligió: T
- Porcentaje de cooperación, y
- Porcentaje de traición

3. Requisitos obligatorios del programa

R1. Funciones

El programa deberá estar organizado utilizando funciones. No se permitirá implementar todo el proyecto dentro de `main()`. Como mínimo deberán existir funciones independientes para:

- Ejecutar una partida.
- Ejecutar el torneo.
- Calcular los puntos.
- Implementar las estrategias.
- Mostrar resultados.

Los estudiantes podrán crear funciones adicionales cuando sean necesarias.

R2. Comentarios

El código deberá contener **comentarios explicativos**. No se requiere comentar cada línea del programa. Los comentarios deberán demostrar que el equipo comprende el código implementado.

R3. Legibilidad

El código deberá presentar:

- Indentación adecuada.
- Nombres descriptivos de variables y funciones.
- Organización lógica.
- Separación adecuada de las funciones.

No se aceptará código innecesariamente complicado o difícil de leer.

R4. Funcionamiento

El programa deberá ejecutarse correctamente y producir resultados coherentes.

Para este proyecto:

- No se utilizarán clases.
- No se utilizará herencia.
- No se utilizará polimorfismo.
- No se requiere interfaz gráfica.
- No se requiere programación concurrente.
- No se utilizarán librerías externas.
- No se requiere conexión a Internet.

El proyecto deberá resolverse utilizando las herramientas de C++ estudiadas durante el curso.

4. Reporte del proyecto

Cada equipo deberá presentar un reporte en PDF que documente el desarrollo del proyecto. El reporte deberá contener las siguientes secciones.

4.1 Portada

Indicar en porcentaje la participación de cada integrante.

4.2 Introducción

4.3 Descripción de las estrategias

Para las dos estrategias propias se deberá indicar:

- Nombre.
- Descripción.
- Reglas.
- Forma en que toma decisiones.

4.4 Diseño del programa

Explicar cómo fue organizado el programa. Se deberá incluir:

- Principales funciones.
- Estructuras de datos utilizadas.
- Variables importantes.
- Organización general del programa.

El reporte deberá incluir un **diagrama de flujo del programa principal**. El diagrama deberá corresponder al programa realmente implementado.

4.5 Análisis de resultados

El equipo deberá analizar los resultados obtenidos. Deberá responder, como mínimo, las siguientes preguntas:

- ¿Cuál fue la estrategia ganadora?
- ¿Por qué consideran que la estrategia ganadora obtuvo el mayor puntaje?
- ¿Contra qué estrategia obtuvo peores resultados? Explicar las posibles razones.
- ¿Las estrategias propias funcionaron como esperaban?

4.6 Conclusiones

5. Entregables

Cada equipo deberá entregar en la plataforma Gradescope:

- Los archivos del programa: **.cpp** y **.h**
- El reporte **en formato PDF**
- No olvide registrar en Gradescope a los integrantes del grupo.

6. Rúbrica

Criterio	Excelente	Adecuado	Mínimo	Insuficiente
1. Diseño del algoritmo	El diseño del algoritmo es correcto y resuelve el problema planteado (6 pts)	Existen algunos errores menores en el diseño del algoritmo (3 pts)	Existen errores en el diseño del algoritmo que afectan parcialmente al resultado (1.5 pt)	Existen muchos errores en el diseño del algoritmo (0pts).
2. Código ⁽¹⁾	El código no muestra errores, es ordenado y corresponde al algoritmo planteado (4 pts)	El código contiene errores menores de programación. (2 pts).	El código contiene errores de programación que afectan parcialmente el resultado (1 pts).	El código contiene múltiples errores de programación (0pts).
3. Presentación escrita (informe) y trabajo en grupo	Describe los puntos del informe en forma ordenada y satisfactoria. Desarrolla el proyecto en forma grupal. Describe las fuentes utilizadas (inclusive la IA) (4 pts)	Muestra errores en la elaboración del informe, no desarrolla el proyecto en forma grupal. Bibliografía incompleta o no se cita correctamente en el informe (2 pts)	Informe esta incompleto, no desarrolla el proyecto en forma grupal. Bibliografía incompleta o inexistente (1 pts)	Informe deficiente, proyecto no es grupal, bibliografía no justifica lo realizado (0pts).
4. Presentación oral	Demuestra dominio del tema en la presentación del proyecto (6 pts)	No demostró un dominio total del contenido del proyecto en algunas partes de la presentación (3 pts).	No demostró un dominio total del contenido del proyecto en gran parte de la presentación (1.5 pts).	No demostró conocimiento del proyecto durante la presentación (0pts).